

Panduan dosen Lab 06 - Compose, PostgreSQL, Redis, dan pemulihan insiden

Durasi demo: 90 menit. **Capaian:** mahasiswa menjalankan API + PostgreSQL + Redis sebagai satu stack, membaca network/volume/secret/healthcheck, menguji cache MISS/HIT, membuktikan data persisten, dan memulihkan gangguan cache. Repo template mandiri: [SeedFlora/meet6CloudService](#). [Modul mahasiswa](#) memuat kunci langkah dan pertanyaan.

Persiapan sebelum kelas

1. Pastikan Docker Desktop/Engine dan Compose plugin hidup: `docker version`, `docker compose version`. Port host 8000 dan 8081 (Adminer opsional) perlu kosong. Lab 05 memakai 8000; hentikan `ccloudlab-api` hanya bila itu container latihan Lab 05 yang sudah selesai.
2. Dari **root repo Lab 06**, salin `secrets/db_password.txt.example` menjadi `secrets/db_password.txt` dan `.env.example` menjadi `.env`. Di PowerShell gunakan `Copy-Item`; di Bash gunakan `cp`. File nyata diabaikan Git. Jangan tampilkan password di proyektor atau screenshot. Jangan ubah password file setelah volume database pertama kali dibuat tanpa prosedur migrasi/reset.
3. Jalankan `docker compose config --quiet`, lalu `docker compose up --build -d --wait`. `--wait` baru kembali setelah healthcheck ketiga service sehat. `docker compose ps` harus menampilkan `api`, `db`, `cache` `healthy`; hanya API mem-publish `127.0.0.1:8000`.
4. Jalankan `Invoke-RestMethod http://127.0.0.1:8000/health` (PowerShell) atau `curl -fsS ...` (Bash). Harapkan `status=ok`, `database=postgres`, `cache=redis`.
5. Simpan data latihan; **jangan** jalankan `docker compose down -v` jika Lab 07 akan memakai database ini. `down` biasa mempertahankan named volume.

Alur 90 menit

Menit	Dosen	Mahasiswa	Bukti/checkpoint
0-12	Gambar alur host → API → DB/Redis, terangkan nama service.	Baca <code>compose.yaml</code> .	Tiga service, network, port hanya API.
12-25	Siapkan file lokal, <code>config --quiet, up --wait</code> .	Ulangi dari repo sendiri.	Tiga container healthy.
25-35	Buka Docker Desktop dan Swagger /docs; <code>getent hosts</code> .	Jelaskan host db/cache vs <code>127.0.0.1</code> .	DNS internal dan port host.
35-50	POST satu note, GET dua kali cepat.	Baca X-Cache dan data.	HTTP 201, MISS lalu HIT.
50-63	<code>psql SELECT, restart api, down/up tanpa -v</code> .	Cocokkan ID/judul.	Data tetap ada.
63-75	Simulasikan <code>stop cache</code> ; baca <code>health/GET</code> ; pulihkan.	Diagnosis 503 versus data 200.	Cache gagal, PostgreSQL tetap sumber data.
75-83	Aktifkan profile Adminer bila waktu cukup.	Lihat tabel <code>notes</code> tanpa password pada bukti.	UI dan API menunjuk DB sama.
83-90	Bahas lima jawaban, Git/secret, cleanup sesuai Lab 07.	Lengkapi laporan dan commit.	Tidak ada <code>.env/password</code> staged.

Peta berkas dan command

`compose.yaml` menetapkan db PostgreSQL, `cache` Redis, `api` FastAPI, dan profile `debug` untuk Adminer. `initdb/01_schema.sql` membuat tabel pada volume DB baru. `api/app/main.py` membaca secret dari `/run/secrets/db_password`, menulis catatan ke PostgreSQL, dan meng-cache daftar di Redis selama 30 detik. `api` hanya mengakses `db/cache` melalui DNS network Compose. Host/browser hanya mengakses API melalui `127.0.0.1:8000`; DB 5432 dan cache 6379 tidak dipublish.

PowerShell persiapan:

```
Copy-Item .\secrets\db_password.txt.example .\secrets\db_password.txt
Copy-Item .\env.example .\env
docker compose config --quiet
docker compose up --build -d --wait
docker compose ps
Invoke-RestMethod http://127.0.0.1:8000/health
```

Jika file lokal sudah ada, jangan menyimpannya tanpa memeriksa volume/kredensial. Di Bash ganti `Copy-Item` dengan `cp`. `docker compose config --quiet` memeriksa bentuk konfigurasi tanpa mencetak secret. `docker compose up ... --wait` membuat image API, network, volume, dan container; dependency `service_healthy` menunggu DB/cache sebelum API start. `docker compose exec api getent hosts db cache` membuktikan DNS internal. `docker compose logs --tail=30 api db cache` memberi jejak startup/HTTP, tanpa perlu `docker exec` ke host.

Demo cache, SQL, dan validasi

PowerShell:

```
$body = '{"title":"Compose","content":"Catatan tetap ada di PostgreSQL"}'
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body $body
(Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing).Headers['X-Cache']
(Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing).Headers['X-Cache']
try { Invoke-WebRequest http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
\'{"title":"  ","content":"uji"}\' -UseBasicParsing } catch { [int]$_ .Exception.Response.StatusCode }
docker compose exec db psql -U clouduser -d cloudnotes -c 'SELECT id,title FROM notes;'
```

POST harus HTTP 201. Dua GET segera sesudah POST menampilkan **MISS** lalu **HIT** sebelum TTL 30 detik habis. POST menghapus cache daftar; GET pertama membaca DB dan mengisi Redis. Judul spasi saja HTTP 422 karena validator memangkas lalu menolaknya. Query SQL memperlihatkan catatan yang sama di sumber kebenaran. Jika cache tidak hidup, GET masih mengambil DB dengan X-Cache: **MISS**, tetapi dapat lebih lambat.

Kunci insiden Redis dan persistensi

Jalankan setelah catatan dibuat. Untuk menunjukkan cache mati:

```
docker compose stop cache
docker compose ps
```

PowerShell membaca gejalanya:

```
try { Invoke-WebRequest http://127.0.0.1:8000/health -UseBasicParsing } catch {
\[int]$_ .Exception.Response.StatusCode }
$r = Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing
$r.StatusCode
$r.Headers['X-Cache']
```

Hasil kunci nyata: health **503** karena cache dependency mati. GET notes tetap **200** dan X-Cache: **MISS**, termasuk note yang sudah dibuat; PostgreSQL masih hidup. Pada uji lokal terbaru, health dan GET masing-masing sekitar **4 detik** ketika DNS/koneksi cache gagal. Kode API melewati percobaan **SETEX** kedua jika GET Redis sudah gagal, sehingga request tidak menunggu dua kali. Ini adalah degradasi yang tetap perlu dicatat, bukan kondisi sehat penuh.

Pulihkan dengan `docker compose start cache`, `docker compose up -d --wait`, lalu GET `/health` 200 dan dua GET `/notes` cepat. Berikutnya:

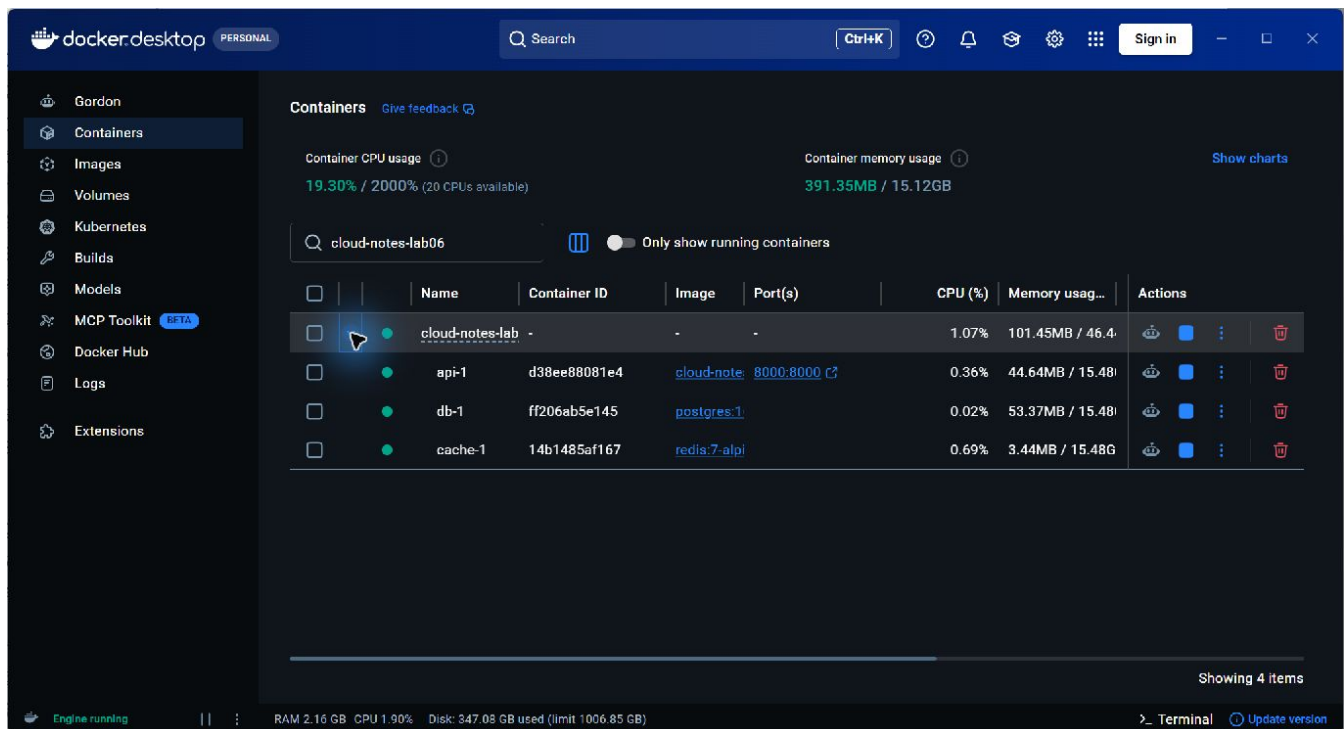
```
docker compose restart api
docker compose --profile debug down
docker compose up -d --wait
docker compose exec db psql -U clouduser -d cloudnotes -c 'SELECT id,title FROM notes;'
docker compose ps
```

`restart api` tidak menghapus volume. `down` menghapus container/jaringan, tetapi **tidak** named volume; `up` membuat container baru yang memasang volume yang sama. Pada uji ini note ID 1 tetap ada setelah restart maupun down/up, dan SQL menunjukkan tiga note total (termasuk note dari uji integrasi lain). Jumlah total dapat berbeda pada kelas.

Jangan tambahkan `-v` kecuali memang ingin menghapus data latihan dan tidak ada lab berikutnya yang memakainya.

Bonus diagnosis isi cache rusak: ketika stack kembali sehat, jalankan `docker compose exec -T cache redis-cli SET notes:list '{broken}'` pada Redis lab ini saja. GET `/notes` pertama harus 200/MISS dari PostgreSQL; API menghapus nilai JSON rusak dan mengisi ulang cache. GET kedua 200/HIT. `/health` tetap 200 karena koneksi Redis sehat. Kasus ini membedakan gangguan **isi cache** dari gangguan **service cache**. Sudah diuji pada stack target, note ID 1 tetap ada.

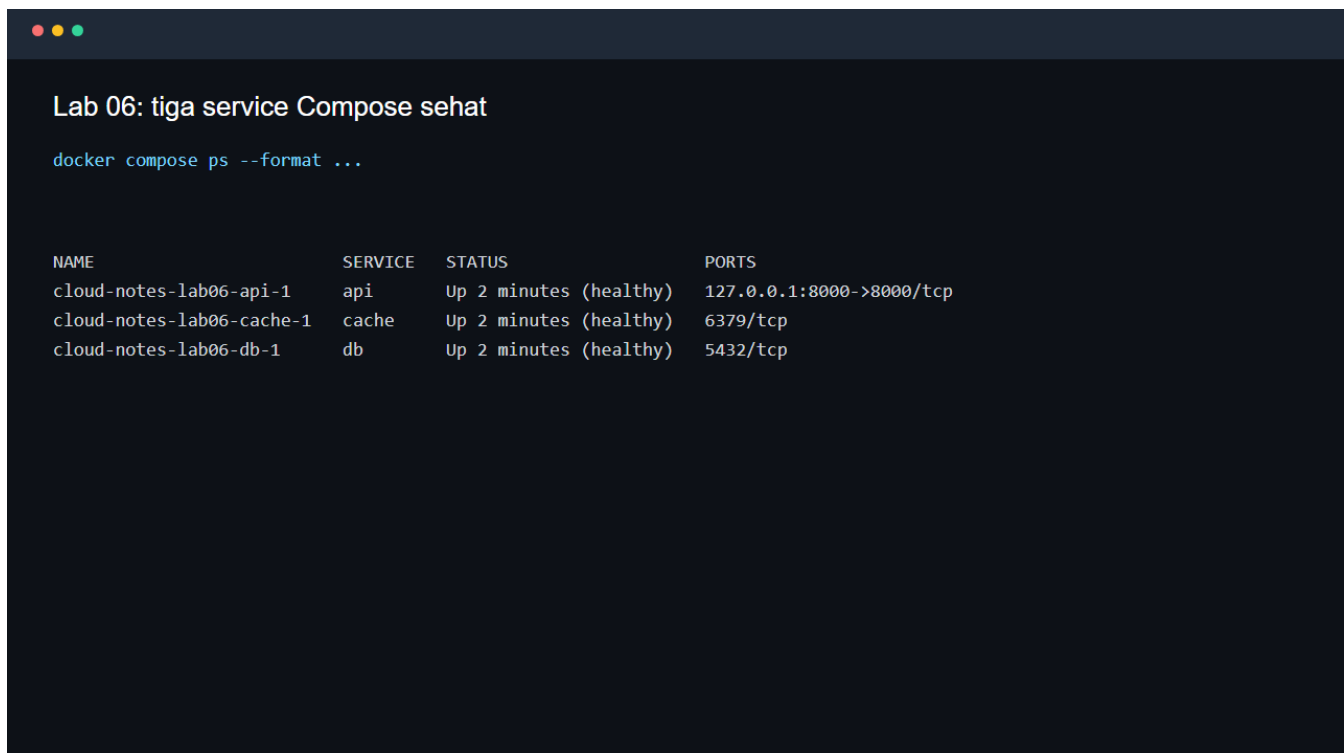
Screenshot dan penjelasan command pada tiap gambar



Docker Desktop dengan tiga container Lab 06 aktif

Perintah: `docker compose up --build -d --wait`, `docker compose ps`, lalu buka Docker Desktop > Containers.

Fungsi: memadankan kondisi CLI dengan UI. **Cara kerja:** Compose menjalankan `api`, `db`, `cache` pada network internal; API saja meneruskan port host 8000. **Baca:** tiga titik hijau berarti container berjalan; terminal harus menulis healthy untuk healthcheck. DB/cache tidak punya mapping port host.



Tiga service pada output Compose awal

Perintah: `docker compose ps`. **Fungsi:** memeriksa service dan port. **Cara kerja:** Compose menanyakan status Docker. **Baca:** `api`, `db`, `cache` healthy; API port 8000.

Cloud Notes 0.6 OAS 3.1

/openapi.json

default

GET	/health	Health	▼
GET	/notes	List Notes	▼
POST	/notes	Create Note	▼
DELETE	/notes/{note_id}	Delete Note	▼

Schemas

HTTPValidationError > Expand all object

NoteInput > Expand all object

ValidationError > Expand all object

OpenAPI dari stack awal

Perintah: buka <http://127.0.0.1:8000/docs>. **Fungsi:** mengenali route aplikasi. **Cara kerja:** FastAPI menyajikan Swagger UI. **Baca:** health, notes, DELETE.

Lab 06: PostgreSQL dan cache Redis

`POST /notes; GET /notes dua kali`

```
POST /notes
{"id":1,"title":"Compose","content":"Data persisten di PostgreSQL","created_at":"2026-09-30T18:13:45.398658+00:00"}

GET /notes, request 1
X-Cache: MISS
[{"id":1,"title":"Compose","content":"Data persisten di PostgreSQL","created_at":"2026-09-30T18:13:45.398658+00:00"}]

GET /notes, request 2
X-Cache: HIT
[{"id":1,"title":"Compose","content":"Data persisten di PostgreSQL","created_at":"2026-09-30T18:13:45.398658+00:00"}]
```

Cache MISS dan HIT

Perintah: POST note, lalu dua GET `/notes`. **Fungsi:** membuktikan Redis dipakai untuk daftar. **Cara kerja:** GET pertama mengambil DB dan mengisi cache; GET kedua membaca Redis. **Baca:** X-Cache: MISS lalu HIT dengan isi data sama.

Cloud Services - hasil perintah lokal

Catatan bertahan di volume PostgreSQL

Output aktual dari sesi uji; ditata ulang agar mudah dibaca pada modul.

POST; SELECT; restart api; GET; down/up; SELECT

POST /notes

```
{ "id":1, "title": "Volume kelas", "content": "Bukti data persisten", "created_at": "2026-10-01T00:56:08.900569+00:00" }
```

SELECT dari PostgreSQL sebelum restart

```
id | title
---+-----
 1 | Volume kelas
(1 row)
```

GET /notes setelah docker compose restart api

```
[{"id":1,"title":"Volume kelas","content":"Bukti data persisten","created_at":"2026-10-01T00:56:08.900569+00:00"}]
```

SELECT sesudah docker compose down / up

```
id | title
---+-----
 1 | Volume kelas
(1 row)
```

Data sebelum dan sesudah restart

Perintah: `psql SELECT, docker compose restart api, down/up, ulangi SELECT`. **Fungsi:** membuktikan volume database. **Cara kerja:** container diganti tetapi db_data tetap. **Baca:** ID/judul sama.

Language: English PostgreSQL > db > cloudnotes > Schema: public clouduser Logout

Adminer 4.17.1 6.1.1

DB: cloudnotes Schema: public

[SQL command](#) [Import](#) [Export](#) [Create table](#) [select notes](#)

Schema: public

[Alter schema](#) [Database schema](#)

Tables and views

Search data in tables (1)

☐	Table	Engine	Collation	Data Length [?]	Index Length [?]	Data Free	Auto Increment	Rows [?]	Comment [?]
☐	notes	table		16,384	16,384	?		?	~ -1
1 in total			en_US.utf8	16,384	16,384	0			

Selected (0)

Move to other database: public

[Create table](#) [Create view](#)

Routines

[Create function](#)

Sequences

[Create sequence](#)

User types

[Create type](#)

Adminer login aman

Perintah: `docker compose --profile debug up -d adminer`, buka `http://127.0.0.1:8081`. **Fungsi:** inspeksi DB lewat UI. **Cara kerja:** Adminer terkoneksi ke hostname db pada network Compose. **Baca:** database cloudnotes; jangan tampilkan password pada bukti.

Language: English PostgreSQL » db » cloudnotes » public » Select: notes clouduser Logout

Adminer 4.17.1 6.1.1

DB: cloudnotes Schema: public

[SQL command](#) [Import](#) [Export](#) [Create table](#)

[select notes](#)

Select: notes

[Select data](#) [Show structure](#) [Alter table](#) [New item](#)

[Select](#) [Search](#) [Sort](#) [Limit](#) [Text length](#) [Action](#)

[SELECT * FROM "notes" LIMIT 50 \(0.001 s\)](#) [Edit](#)

☐ Modify	id	title	content	created_at
☐ edit	1	Volume kelas	Bukti data persisten	2026-10-01 00:56:08.900569+00

Whole result ☐ 1 row [Modify](#) [Save](#) [Selected \(0\)](#) [Export \(1\)](#)

[Import](#)

Tabel notes di Adminer

Perintah: pilih tabel `notes` dan **Select data**. **Fungsi:** membandingkan hasil API/SQL/UI. **Cara kerja:** Adminer menjalankan SELECT ke PostgreSQL yang sama. **Baca:** ID/judul sama; jangan klik Drop/Truncate.

```
Lab 06 - Compose/API aktual
docker compose up --build -d --wait -> 3 service healthy
GET /health -> HTTP 200, database=postgres, cache=redis
POST /notes -> HTTP 201, id=1, title=Audit kelas Lab06
GET /notes pertama -> HTTP 200, X-Cache: MISS
GET /notes kedua -> HTTP 200, X-Cache: HIT
POST title=" " -> HTTP 422
```

Output Docker/API aktual ditata ulang agar terbaca; secret tidak direkam.

Uji ulang cache dan validasi

Perintah: `up --wait`, POST, dua GET, POST judul spasi. **Fungsi:** bukti run terbaru. **Cara kerja:** output Docker/API aktual ditata agar terbaca. **Baca:** healthy, 201, MISS→HIT, 422.

Cloud Notes 0.6 OAS 3.1

/openapi.json

default

GET	/health	Health	⌵
GET	/notes	List Notes	⌵
POST	/notes	Create Note	⌵
DELETE	/notes/{note_id}	Delete Note	⌵

Schemas

HTTPValidationError > Expand all object

NoteInput > Expand all object

ValidationError > Expand all object

OpenAPI yang berjalan kembali

Perintah: buka `/docs` pada port 8000. **Fungsi:** membuktikan route live, termasuk DELETE. **Cara kerja:** Chrome lokal memuat halaman API. **Baca:** versi 0.6 dan endpoint yang tersedia.

```
Lab 06 - incident Redis dan persistensi, uji lokal aktual
docker compose stop cache
GET /health -> HTTP 503 (4.07 detik)
GET /notes -> HTTP 200, X-Cache: MISS (4.0 detik)
Catatan id=1 tetap ada di PostgreSQL
docker compose start cache -> healthy
docker compose restart api -> catatan id=1 tetap ada
docker compose --profile debug down; docker compose up -d --wait
Catatan id=1 tetap ada; SELECT count(*) = 3
docker compose ps -> api, db, cache healthy
```

Transkrip Docker/API aktual ditata ulang; tidak ada volume yang dihapus.

Insiden cache dan data persisten

Perintah: stop cache, cek health/notes, start cache, restart API, down/up tanpa `-v`, query SQL. **Fungsi:** bukti diagnosis dan pemulihan. **Cara kerja:** catatan tetap di volume PostgreSQL saat service cache/API berganti. **Baca:** health 503 saat outage, GET 200/MISS, ID tetap ada sesudah pemulihan. Transkrip aktual ditata ulang; angka waktu dapat berbeda.

```
Lab 06 - cache Redis berisi JSON rusak (uji aktual)
docker compose exec -T cache redis-cli SET notes:list "{broken}" -> OK
GET /notes -> HTTP 200, X-Cache: MISS
ID catatan tetap ada dari PostgreSQL
GET /notes ulang -> HTTP 200, X-Cache: HIT
GET /health -> HTTP 200
```

Transkrip Docker/API aktual ditata ulang; cache rusak diganti dari sumber PostgreSQL.

Cache JSON rusak dipulihkan dari DB

Perintah: `docker compose exec -T cache redis-cli SET notes:list '{broken}'` dan dua `GET /notes`.

Fungsi: menguji ketahanan saat data cache rusak. **Cara kerja:** API menangkap error parse, membuang cache rusak, mengambil PostgreSQL, lalu menulis salinan sehat. **Baca:** 200/MISS di request pertama, 200/HIT di kedua; health tetap 200 karena Redis hidup.

Kunci analisis, masalah umum, dan pengumpulan

1. `DB_HOST=db` berlaku dalam DNS Compose; browser host memanggil `127.0.0.1:8000` yang dipublish API.
2. `restart api` mengganti proses; `down` mengganti container/jaringan; volume tetap. `down -v` juga menghapus volume dan data latihan.
3. `POST/DELETE` menghapus cache daftar; `GET` pertama MISS dan mengisi Redis selama 30 detik; `GET` berikutnya HIT sampai TTL habis atau data berubah.
4. `depends_on: service_healthy` menunda startup awal API, tetapi tidak menjamin dependency tidak akan gagal kemudian. Saat Redis mati, health 503 dan `GET` fallback ke DB.
5. `.env.example` dan secret contoh boleh ada di Git; `.env` dan `secrets/db_password.txt` nyata harus lokal. Cek `git check-ignore -v .env secrets/db_password.txt` di repo pribadi sebelum commit.

Gejala	Diagnosis/aksi
Port 8000 dipakai	<code>docker ps</code> ; hentikan container Lab 05 sendiri bila selesai.
API masih <code>starting</code>	Gunakan <code>up --wait</code> , baca <code>docker compose logs --tail=50 api db cache</code> .
Password DB gagal setelah file diubah	Volume lama menyimpan password lama; jangan hapus volume yang berisi pekerjaan. Cocokkan kredensial atau lakukan reset data latihan dengan izin jelas.
Dua <code>GET</code> sama-sama MISS	Jalankan berurutan dalam 30 detik; <code>POST</code> di antaranya memang menghapus cache.
Health 503 tetapi notes 200	Cache mungkin mati; cek <code>docker compose ps</code> dan log.
Adminer masih hidup setelah cleanup	Gunakan <code>docker compose --profile debug down</code> .

Laporan `hasil/lab06.md` dibuat dari `hasil/TEMPLATE_LAPORAN.md`, bukti pribadi di `hasil/bukti/`. Cek `git status`, `git diff --check`, `git add compose.yaml api initdb secrets .env.example hasil/lab06.md hasil/bukti`, `git diff --cached --name-only`, dan `git diff --cached --check` sebelum commit/push. Jangan menempelkan password di SQL/screenshot. **Teruji pada komputer ini:** config Compose, build, tiga service healthy, OpenAPI browser, HTTP health/POST/GET/422, MISS/HIT, fallback saat Redis mati, cache JSON rusak, pemulihan, restart API, down/up tanpa volume hapus, SQL persistence. Stack dikembalikan healthy setelah uji.